

EXERCISE SOLUTIONS



Module 1: Introduction to Databases & RDBMS Fundamentals

Module 1 - 6 Hours

Introduction to Databases & RDBMS Fundamentals

Notes for the trainer

Core and stretch. The questions marked **(Core)** are the required set that fits the session. Debrief those with the whole room. Cover the unmarked stretch questions only if the batch worked ahead, or leave them as take-home reading.

Release this file only after the practice window closes. Marks are indicative and total 100.

Section	Questions	Marks
Level 1 - Understanding	Q1 to Q10	30
Level 2 - Applying SQL	Q11 to Q22	45
Level 3 - Analysis	Q23 to Q29	25

Where a query's output depends on the seeded data, the expected numbers are given. If a learner's numbers differ, they loaded the seed file twice.

Level 1 - Understanding

Q1 (Core)

Answer. The two problems are **data redundancy** (the same address stored in two places) and **data inconsistency** (the two copies now disagree, and nothing says which is correct).

A database prevents both by storing the address exactly once, in `customer.city` and the address columns, and having the credit card system read it from there rather than keep a copy. There is then only one value to update, so it cannot disagree with itself.

Marking: 1 mark per named problem, 1 for the single-source-of-truth explanation.

Q2 (Core)

Answer. Any three of:

DBMS capability	Banking consequence if missing
Concurrency control	Two tellers post to the same account and one update is silently lost
Integrity enforcement	A balance of -900,000 is accepted and reaches the general ledger
Backup and recovery	A disk failure loses a day of transactions with no way to reconstruct them
Security and authorisation	A junior clerk can read every salary account in the branch
A query language	Every new report needs a developer to write file-scanning code
Transactions	A transfer debits one account and never credits the other

Q3

Formal term	Everyday term	ICICI Bank example
Relation	Table	<code>account</code>
Tuple	Row	One savings account belonging to customer 1001
Attribute	Column	<code>balance</code>
Degree	Number of columns	<code>account</code> has degree 10
Cardinality	Number of rows	<code>account</code> has cardinality 36

Q4 (Core)

Column	Answer	Why
<code>customer.customer_id</code>	Primary key	Declared <code>pk_customer</code> , unique and not null
<code>account.account_number</code>	Candidate key	Unique, but <code>account_id</code> was chosen as the primary key
<code>account.customer_id</code>	Foreign key	Must match <code>customer.customer_id</code>
<code>account.balance</code>	None	Not unique and references nothing
<code>employee.manager_id</code>	Foreign key	Self-referencing, points at <code>employee.employee_id</code>

Column	Answer	Why
<code>branch.ifsc_code</code>	Candidate key	Unique, but <code>branch_id</code> is the primary key

Q5 (Core)

Workload	Type	Reason
Balance check on the mobile app	OLTP	Single row, sub-second, very high frequency
Quarterly NPA by branch	OLAP	Aggregates over millions of loan and payment rows
ATM dispenses 10,000	OLTP	Short write transaction that must complete immediately
Regulator's yearly transaction analysis	OLAP	Full-year scan, read only, run rarely
Teller opens a savings account	OLTP	A few inserts, interactive response needed
Top 100 customers by average balance	OLAP	Aggregation and ranking across all accounts

Q6

Answer. Three technical reasons:

1. A full scan of `account_txn` evicts hot data pages from the buffer cache, so ordinary customer queries start reading from disk and slow down for everyone.
2. The report holds read locks and long-running snapshots, which on Oracle can cause snapshot-too-old errors and on any engine delays cleanup of old row versions.
3. Report queries consume CPU and I/O that the ATM and UPI workload needs to meet its sub-second response target. The core banking database has no spare capacity budgeted for it.

Offer instead: run the report against a read replica, or against the SQL Server reporting warehouse loaded overnight. If the freshness genuinely matters, offer a replica with a few seconds of lag rather than the primary.

Q7 (Core)

Atomicity - **d**. Consistency - **c**. Isolation - **b**. Durability - **a**.

Q8 (Core)

Answer. The log record is small and written sequentially, so it reaches disk quickly, while the data page is large and randomly located. Writing the log first means that at every instant the log describes at least as much as the data files contain.

If the order were reversed, a crash could leave a data page already changed on disk with no log record describing the change. Recovery would have no way to know whether that change belonged to a committed transaction or an abandoned one, so it could neither safely keep it nor safely undo it. The database would

be unable to guarantee either atomicity or durability.

Q9

Answer. It breaks **logical data independence**. `SELECT *` binds the application to the current column list, so a change to the logical design leaks into application code.

When `pan_number` is added, every row returned gains an extra column. Code that reads results by position now reads the wrong field, object mappers may fail to bind an unknown column, and sensitive data is shipped to a screen that was never meant to display it. Naming the columns explicitly means the new column changes nothing.

Q10

Product	Where at ICICI Bank	Why
Oracle	Core banking: accounts, transactions, general ledger	RAC availability, partitioning for very large tables, mature recovery tooling, and it came with the core banking product
PostgreSQL	New digital lending platform	No licence cost, strong standard SQL, JSONB for document payloads
MySQL	Internet banking session and profile store	Simple, fast for read-heavy key lookups, easy to replicate
Redis	Balance and profile cache in front of the core	In-memory, sub-millisecond, removes repeated identical reads from the database

Level 2 - Applying SQL

Q11

```
SELECT current_database(), current_user, inet_server_port();
```

```
current_database | current_user | inet_server_port
-----+-----+-----
icicibank       | postgres    |                5432
```

Q12

```
SELECT column_name, data_type, is_nullable
FROM   information_schema.columns
WHERE  table_name = 'loan'
ORDER BY ordinal_position;
```

```
column_name | data_type | is_nullable
-----+-----+-----
loan_id     | integer  | NO
customer_id | integer  | NO
branch_id   | integer  | NO
loan_type   | character varying | NO
principal_amount | numeric | NO
interest_rate | numeric | NO
tenure_months | integer | NO
emi_amount   | numeric  | NO
disbursed_date | date    | NO
outstanding_amount | numeric | NO
status       | character varying | NO
```

`loan` has **11 columns**, so the degree of the relation is **11**. Every column is **NOT NULL**, which is a deliberate design decision: a loan with an unknown principal or tenure is not a loan.

Q13 (Core)

```
SELECT 'account' AS table_name, COUNT(*) AS row_count FROM account
UNION ALL SELECT 'account_txn', COUNT(*) FROM account_txn
UNION ALL SELECT 'beneficiary', COUNT(*) FROM beneficiary
UNION ALL SELECT 'branch', COUNT(*) FROM branch
UNION ALL SELECT 'card', COUNT(*) FROM card
UNION ALL SELECT 'customer', COUNT(*) FROM customer
UNION ALL SELECT 'employee', COUNT(*) FROM employee
UNION ALL SELECT 'loan', COUNT(*) FROM loan
UNION ALL SELECT 'loan_payment', COUNT(*) FROM loan_payment
ORDER BY table_name;
```

```
table_name | row_count
-----+-----
account    |        36
```

```

account_txn |      495
beneficiary |      60
branch      |       8
card        |      32
customer    |      30
employee    |      24
loan        |      12
loan_payment |     144

```

Common mistake: writing nine separate `SELECT COUNT(*)` statements. The question asked for a single result set, which is what `UNION ALL` is for. Note that `ORDER BY` applies to the whole union and must appear once, at the end.

Q14

```

SELECT tc.table_name AS child_table,
       kcu.column_name AS child_column,
       ccu.table_name AS parent_table,
       ccu.column_name AS parent_column
FROM   information_schema.table_constraints tc
       JOIN information_schema.key_column_usage kcu
         ON tc.constraint_name = kcu.constraint_name
       JOIN information_schema.constraint_column_usage ccu
         ON ccu.constraint_name = tc.constraint_name
WHERE  tc.constraint_type = 'FOREIGN KEY'
ORDER BY tc.table_name, kcu.column_name;

```

Ten foreign keys are returned. The self-reference is:

```

child_table | child_column | parent_table | parent_column
-----+-----
employee    | manager_id   | employee     | employee_id

```

What it represents: the branch reporting hierarchy. A Relationship Manager's `manager_id` holds the `employee_id` of their Branch Manager. A Branch Manager has `manager_id` set to NULL, which is why that column is the only nullable foreign key in the schema. You will query this with a self join in Module 3 and with a recursive CTE in Module 6.

Q15

```

SELECT relname AS table_name,
       pg_size_pretty(pg_total_relation_size(relid)) AS total_size
FROM   pg_catalog.pg_statio_user_tables
ORDER BY pg_total_relation_size(relid) DESC
LIMIT 3;

```

```

table_name | total_size
-----+-----
account_txn | 136 kB
beneficiary | 48 kB
customer    | 48 kB

```

`account_txn` is largest because it is the only table with a row per **event** rather than a row per **entity**. There are 30 customers and 36 accounts, but 495 transactions, and in production that ratio is far more extreme: 30,000 customers against 2 million transactions a month. Transaction tables are always the biggest object in a core banking database, which is why Modules 5 and 7 spend their effort there.

Q16 (Core)

```
INSERT INTO customer
(customer_id, first_name, last_name, date_of_birth, email, phone,
city, kyc_status, risk_rating, customer_since, home_branch_id)
VALUES
(1032, 'Latha', 'Krishnan', DATE '1985-09-23', 'latha.krishnan@example.com',
'9845099887', 'Chennai', 'VERIFIED', 'MEDIUM', DATE '2026-03-01', 5);

INSERT INTO account
(account_id, account_number, customer_id, branch_id, account_type,
balance, currency, status, opened_date, interest_rate)
VALUES
(9047, 'ICIC0000009047', 1032, 5, 'CURRENT',
500000.00, 'INR', 'ACTIVE', DATE '2026-03-01', 0.00);

SELECT c.customer_id, c.first_name, c.last_name, a.account_number,
a.account_type, a.balance
FROM customer c JOIN account a ON a.customer_id = c.customer_id
WHERE c.customer_id = 1032;
```

customer_id	first_name	last_name	account_number	account_type	balance
1032	Latha	Krishnan	ICIC0000009047	CURRENT	500000.00

Order matters. The customer must be inserted first. Reversing the two statements produces a foreign key violation on `fk_account_customer_id`.

Q17 (Core)

```
UPDATE account SET balance = balance + 75000 WHERE account_id = 9047;
UPDATE account SET balance = balance - 20000 WHERE account_id = 9047;

SELECT account_number, balance FROM account WHERE account_id = 9047;
```

account_number	balance
ICIC0000009047	555000.00

Why relative and not absolute. `SET balance = 555000` assumes nothing else changed the row between your read and your write. If a UPI credit landed in between, your absolute value silently erases it. `balance = balance + 75000` is computed by the engine while it holds the row lock, so no update can be lost. This is the difference between a correct banking system and one that loses money under load.

Q18 (Core)

```
DELETE FROM customer WHERE customer_id = 1032;
```

```
ERROR: update or delete on table "customer" violates foreign key constraint
       "fk_account_customer_id" on table "account"
DETAIL: Key (customer_id)=(1032) is still referenced from table "account".
```

Correct order:

```
DELETE FROM account WHERE account_id = 9047;
DELETE FROM customer WHERE customer_id = 1032;

SELECT (SELECT COUNT(*) FROM customer) AS customers,
       (SELECT COUNT(*) FROM account) AS accounts;
```

```
customers | accounts
-----+-----
        30 |         45
```

The database is protecting itself from an **orphan row**: an account belonging to a customer who no longer exists. Module 4 shows the alternative, `ON DELETE CASCADE`, and explains why you would almost never enable it on a customer table in a bank.

Q19

1. Invalid account type

```
INSERT INTO account
(account_id, account_number, customer_id, branch_id, account_type,
 balance, currency, status, opened_date, interest_rate)
VALUES (9099, 'ICIC0000009099', 1001, 1, 'FIXED', 1000, 'INR', 'ACTIVE',
       DATE '2026-03-01', 0);
```

```
ERROR: new row for relation "account" violates check constraint "ck_account_1"
DETAIL: Failing row contains (9099, ICIC0000009099, 1001, 1, FIXED, ...).
```

2. Negative transaction amount

```
INSERT INTO account_txn
(txn_id, account_id, txn_date, txn_type, amount, channel, narration, status, reference_no)
VALUES (999998, 9001, TIMESTAMP '2026-03-01 10:00:00', 'DEBIT', -5000,
       'ATM', 'test', 'SUCCESS', 'REF999998');
```

```
ERROR: new row for relation "account_txn" violates check constraint "ck_account_txn_2"
```

A negative amount is meaningless. Direction is carried by `txn_type`, not by the sign of `amount`. Allowing both would let the same movement be expressed two ways, and every report would have to handle both.

3. Invalid KYC status

```
INSERT INTO customer
(customer_id, first_name, last_name, date_of_birth, email, phone,
city, kyc_status, risk_rating, customer_since, home_branch_id)
VALUES (1033, 'Test', 'User', DATE '1990-01-01', 'test.user@example.com',
'9800000000', 'Pune', 'IN_PROGRESS', 'LOW', DATE '2026-03-01', 7);
```

```
ERROR: new row for relation "customer" violates check constraint "ck_customer_1"
```

`kyc_status` is restricted to PENDING, VERIFIED and REJECTED. This is a **domain constraint**: the column's set of legal values. If the bank genuinely needs an IN_PROGRESS state, the fix is to change the constraint deliberately, not to let arbitrary strings in.

Q20

Session A:

```
BEGIN;
UPDATE account SET balance = balance - 5000 WHERE account_id = 9002;
SELECT account_id, balance FROM account WHERE account_id = 9002;
```

```
account_id | balance
-----+-----
9002      | 38522.54
```

Session B, at the same time:

```
SELECT account_id, balance FROM account WHERE account_id = 9002;
```

```
account_id | balance
-----+-----
9002      | 43522.54
```

Session A:

```
ROLLBACK;
```

Session B again: the balance is still 43522.54.

Session B never saw the uncommitted value. Reading another transaction's uncommitted change is called a **dirty read**, and PostgreSQL does not permit it at any isolation level. Session B was not blocked either, because it was only reading - MVCC gave it the last committed version of the row.

Q21 (Core)

```
SELECT account_id, balance FROM account WHERE account_id IN (9004, 9010);

BEGIN;
UPDATE account SET balance = balance - 12000 WHERE account_id = 9004;
UPDATE account SET balance = balance + 12000 WHERE account_id = 9010;
SELECT account_id, balance FROM account WHERE account_id IN (9004, 9010);
ROLLBACK;
```

```
SELECT account_id, balance FROM account WHERE account_id IN (9004, 9010);
```

The middle SELECT shows both balances changed. The final SELECT shows both back to their original values, identical to the first SELECT. Nothing survived, because **ROLLBACK** discards the entire unit of work. That is **atomicity**.

Common mistake: forgetting **BEGIN**. Without it PostgreSQL runs each statement in its own auto-committed transaction, the money really moves, and **ROLLBACK** reports a warning that there is no transaction in progress.

Q22

```
SHOW shared_buffers;  
SHOW work_mem;  
SHOW wal_level;  
SHOW fsync;
```

```
shared_buffers | 128MB  
work_mem       | 4MB  
wal_level      | replica  
fsync          | on
```

Setting	What it controls
<code>shared_buffers</code>	The size of the shared cache of data pages. Pages found here are served without touching disk.
<code>work_mem</code>	Memory each sort or hash operation may use before spilling to a temporary file on disk.
<code>wal_level</code>	How much detail goes into the write-ahead log. <code>replica</code> includes enough for a standby server, which Module 9 uses.
<code>fsync</code>	Whether the engine forces the log to physical disk before acknowledging a commit. <code>on</code> is what makes durability real.

Level 3 - Analysis and design decisions

Q23

Model answer.

MyISAM must not be used. It supports neither transactions nor foreign keys, and it locks at table level rather than row level.

The specific risk: a UPI payment is at minimum a debit, a credit and a ledger insert. On MyISAM those are three independent writes with no way to roll back. A crash or an error between them leaves the bank's books wrong, and there is no mechanism to detect or repair it other than manual reconciliation. Table-level locking also makes the "faster for reads" claim wrong at 4,000 writes per second, because every write blocks the whole table.

The alternative: **InnoDB**, which is MySQL 8's default engine and provides row-level locking, foreign keys and full ACID transactions. If the vendor's benchmark showed MyISAM winning, it was measuring a read-only workload that does not resemble UPI. For this throughput I would also want the design reviewed against PostgreSQL or the existing Oracle core before committing.

Marking: 3 marks for naming the transaction risk, 2 for the locking point, 2 for naming InnoDB, 1 for challenging the benchmark.

Q24

Model answer.

The two workloads want opposite things. The branch screen is OLTP: it needs one account's transactions in milliseconds, which is served by a normalized table with a narrow index on (`account_id`, `txn_date`). The risk team's query is OLAP: it scans every row for a month across all accounts, which wants denormalized, pre-aggregated, column-oriented storage. Tuning the same table for both means indexes that slow down inserts and still do not make the scan fast, and the scan itself evicts the OLTP data from the buffer cache.

The arrangement I would propose is two systems. The core banking database keeps `account_txn` normalized and indexed for point lookups, serving the branch screen. A separate reporting database holds the same data reshaped for analysis, loaded overnight, serving the risk team. Data moves between them on a schedule, and the reporting system is allowed to be a day behind.

Part	Built in
Indexing and tuning the OLTP access path	Module 5
Caching the hottest lookups in Redis	Module 7
Replication and monitoring the two servers	Module 9
The SQL Server reporting warehouse	Module 10
The ETL pipeline that loads it	Modules 10 and 11

Q25 (Core)

Model answer.

The application log records what the application believed, not what the database committed. The application writes that line after its own call returns, and there are several ways for the call to return without a durable commit: the transaction was never committed and was rolled back when the connection closed, the commit failed and the exception was swallowed, an ORM held the work in a session that was later discarded, or the connection dropped after the statement but before the commit.

The evidence would be the database's own record: the row exists in `account_txn` with the expected reference number, both balances reflect the movement, and the commit appears in the transaction log for that timestamp. In practice you reconcile the application's reference number against the database, and where the two disagree, the database is right.

Q26

Model answer.

Not on the core banking server, under any circumstances. `fsync = off` means PostgreSQL acknowledges a commit before the log record is physically on disk. An operating system crash or power loss can then lose transactions that the bank has already told a customer succeeded, and worse, it can leave the data files internally inconsistent so the database will not start at all. The bank would be unable to state its own position.

Where it is legitimate: a throwaway environment where losing everything is acceptable and the data can be regenerated. Loading a training database, running an automated test suite in CI, or a scratch instance for benchmarking. It must never be enabled on a server that holds the only copy of anything.

For the actual problem, faster overnight loads, the safe options are `COMMIT` less often by batching rows into larger transactions, raising `max_wal_size` to reduce checkpoint frequency, using `COPY` instead of row-by-row inserts, dropping and rebuilding indexes around the load, and putting the WAL on a separate fast disk. Those get most of the benefit and keep the guarantee.

Q27 (Core)

Model answer.

Two arguments for the database:

1. The database is the only layer every path goes through. The bank has a mobile app, an internet banking site, an ATM switch, batch jobs and DBAs running manual corrections. A rule in one application is enforced for one of those paths. A rule in the database is enforced for all of them, including the correction script someone runs at 2 a.m. during an incident.
2. Constraints are also documentation and are checked against data already present. When a constraint is added, the engine validates existing rows, so it surfaces corruption that has already happened. Application code cannot do that retrospectively.

Where the application layer is right: rules that need context the database does not have. Deciding whether a transfer exceeds a customer's daily limit given their KYC tier, product entitlements and fraud score involves data from several systems and business policy that changes weekly. Putting that in a check constraint would be unmaintainable. The rule of thumb is that invariants of the data belong in the database, and decisions about the business process belong in the application.

Q28

Model answer. Four expected sources of trouble:

1. **Data type mismatches.** Oracle `VARCHAR2` and `NUMBER` do not map cleanly onto PostgreSQL `VARCHAR` and `NUMERIC`. Oracle's `NUMBER` with no precision, and its treatment of an empty string as `NULL`, both produce surprises during a load.
2. **Schema and user meaning.** In Oracle a schema is a user; in PostgreSQL a database contains many schemas and users are separate objects. The object naming and permission model has to be redesigned, not copied.
3. **Procedural code.** PL/SQL packages do not run on PostgreSQL. PL/pgSQL is similar in spirit but different in syntax, and packages have no direct equivalent. Every stored procedure in the acquired bank's system is a rewrite.
4. **Sequences, identity and defaults.** Oracle sequences with `NEXTVAL` in triggers versus PostgreSQL identity columns and `SERIAL` behave differently, and getting it wrong silently produces duplicate or skipped keys.

Also creditable: case folding of unquoted identifiers, date and timestamp handling with time zones, `DUAL`, `ROWNUM` versus `LIMIT`, `MINUS` versus `EXCEPT`, and differences in default isolation and locking behaviour.

Q29

Model answer. One acceptable version:

Dear Ms Rao,

Thank you for raising ICICI-4417. Here is what moving branch balances off spreadsheets and into a shared database will change.

Today each branch keeps its own copy of a customer's balance, so when money moves between Fort Mumbai and Andheri East there are two numbers that have to be corrected by hand, and until someone does that, the two branches disagree. With a shared database there is only one balance for each account. Both branches read and update the same number, so there is nothing to reconcile - the disagreement cannot arise in the first place. Two further things change. First, a transfer becomes a single piece of work: either both the debit and the credit happen, or neither does. There is no state where money has left one account and not arrived at the other, even if a server fails midway. Second, the system will refuse impossible data. A balance cannot go below zero, and an account cannot be created for a customer who does not exist. Today a spreadsheet accepts both.

What this will not fix: it will not correct the balances that are already wrong. We will need a one-time reconciliation before we switch over, and Operations will have to sign off the opening figures. It also will not remove the need for a daily check - it changes it from three days of manual matching to an automated report we read in the morning.

What has to change on your side: branch staff will no longer be able to edit balances directly. Corrections will go through a maker-checker process with an audit trail. That is deliberate, and it is what makes the single balance trustworthy.

Marking: 5 marks. Look for the single-source-of-truth explanation, the all-or-nothing transfer explained without the word atomicity, at least one honest limitation, and a real operational consequence for the branch. Deduct for using the forbidden vocabulary.